School of Computing

SRM Institute of Science and Technology, Kattankulathur – 603 203

Department of Networking & Communications | BTech CSE (Cyber Security)

ARCHITECTURE DOCUMENT

KaliMind

Autonomous Vulnerability Assessment Agent

LLM-Assisted Penetration Testing via Model Context Protocol (MCP)

Team KaliMind

BTech CSE (Cyber Security) | Dept. of Networking & Communications

Srikar Bharat (RA2311030010227) • R Aswin (RA2311030010258) • Deepthi Chand Mungara (RA2311030010260)

Faculty Guide & Panel Head: Dr. C.N.S. Vinoth Kumar

Panel Members: Dr. C.N.S. Vinoth Kumar • Dr. P. Balaji Srikaanth • Dr. R. Nithya • Ms. Jasmine S

Source: kalimind_backend.py · kalimind_mcp.py · kalimind.json

1. Architecture Selection & Justification

1.1 Methodology Considered

Four primary architectural methodologies were evaluated: Monolithic, Microservices, Event-Driven, and Serverless. Selection criteria were driven by the system's requirements — LLM–tool integration via HTTP REST, Kali Linux subprocess execution, subprocess isolation, and real-time output streaming.

Criterion	Monolithic	Microservices	Event-Driven	Serverless
Deployment Complexity	Low	Medium	High	Low
Scalability	Limited	High	High	Auto
LLM–Tool Integration	Difficult	Natural (REST)	Complex	Limited
Subprocess Isolation	Poor	Per-service	Queue-based	Function-level
Kali VM Compatibility	Poor	Excellent	Moderate	Poor
Fault Isolation	None	Strong	Strong	Moderate
KaliMind Best Fit?	No	YES ✓	No	No

1.2 Selected Architecture: Microservices

KaliMind implements a two-service microservices architecture, as confirmed by the actual codebase:

- Service 1 — MCP Client: kalimind_mcp.py runs on the operator's local machine alongside Claude Desktop. Implements FastMCP server via the mcp.server.fastmcp library. Registers 11 MCP tools. Communicates with the backend via HTTP using the requests library (timeout: 300s).
- Service 2 — Flask Backend: kalimind_backend.py runs on the Kali Linux VM. Implements a Flask REST API on port 5000 (default bind: 127.0.0.1). Executes tool commands via the CommandExecutor class using subprocess.Popen(shell=True). Backend timeout: 180 seconds.

1.3 Justification

- LLM–Tool Separation: The MCP layer translates LLM tool_use calls into HTTP REST requests. The LLM never has direct filesystem or shell access — only well-defined JSON API endpoints are exposed.
- Subprocess Isolation: Each tool invocation creates an independent CommandExecutor instance with its own subprocess. Separate daemon threads read stdout and stderr continuously, preventing buffer deadlock. A hung tool times out at 180s (SIGTERM → SIGKILL), returning partial results if any output was captured.
- Dual Timeout Architecture: Backend: COMMAND_TIMEOUT = 180s. MCP client: DEFAULT_REQUEST_TIMEOUT = 300s. This ensures the backend kills a runaway process before the client times out waiting for the HTTP response.
- Configurable Deployment: The backend IP binding is configurable via the --ip argument (default: 127.0.0.1). The port is configurable via --port (default: 5000). The MCP client target server is configurable via --server (default: http://localhost:5000).

- Extensibility: New tools are added by registering a new Flask route in kalimind_backend.py and a new @mcp.tool() decorator function in kalimind_mcp.py — no core refactoring required.

2. Source Code Architecture

2.1 kalimind_backend.py — Flask Execution Engine

Property	Details
Runtime	Python 3.x — Flask, subprocess, threading, argparse, logging
Default Bind	127.0.0.1:5000 (configurable via --ip and --port flags)
Key Class	CommandExecutor — manages subprocess lifecycle, stdout/stderr threads, timeout, SIGTERM→SIGKILL
Timeout (Backend)	COMMAND_TIMEOUT = 180 seconds
Debug Mode	Controlled by DEBUG_MODE env var or --debug flag; sets Flask debug=True
Registered Endpoints	13 endpoints (see Section 3)
Stub Endpoints	/mcp/capabilities and /mcp/tools/kalimind_tools/<tool_name> — Sprint 4 scope
Metasploit Special Case	Auto-generates resource script at /tmp/mcp_msf_resource.rc; cleaned up after execution
Partial Results	If timed_out=True AND stdout/stderr has data → success=True, partial_results=True

2.2 CommandExecutor Class (kalimind_backend.py)

The CommandExecutor class handles all subprocess execution. Key behaviour:

- Spawn: subprocess.Popen(shell=True, stdout=PIPE, stderr=PIPE, text=True, bufsize=1)
- Streaming: Two daemon threads (_read_stdout, _read_stderr) continuously read output lines, preventing pipe buffer blocking for long-running tools
- Timeout Handling: process.wait(timeout=180) → on TimeoutExpired: terminate() → wait 5s → kill() if still running
- Return Value: Always returns { stdout, stderr, return_code, success, timed_out, partial_results }
- Success Logic: success=True if return_code==0, OR if timed out but produced output (partial_results=True)

```
terminal
# CommandExecutor response schema (kalimind_backend.py)
{
  "stdout":          str,   # Captured standard output
  "stderr":          str,   # Captured standard error
  "return_code":     int,   # Exit code (-1 on timeout/error)
  "success":         bool,  # True if clean exit OR partial timeout output
  "timed_out":       bool,  # True if CommandExecutor timeout triggered
  "partial_results": bool   # True if timed_out AND output was captured
}
```

2.3 kalimind_mcp.py — MCP Client Bridge

Property	Details
Runtime	Python 3.x — mcp.server.fastmcp (FastMCP), requests, argparse, logging
MCP Server Name	kali_mcp
Default Backend URL	http://localhost:5000 (configurable via --server flag)
Timeout (MCP Client)	DEFAULT_REQUEST_TIMEOUT = 300 seconds
Registered MCP Tools	12 tools: nmap_scan, gobuster_scan, dirb_scan, nikto_scan, sqlmap_scan, metasploit_run, hydra_attack, john_crack, wpscan_analyze, enum4linux_scan, execute_command, server_health
Health Check on Start	Calls GET /health on startup; warns if tools missing but starts anyway
HTTP Methods Used	safe_get() for /health; safe_post() for all tool endpoints
Logging	Logs to stderr at INFO level; --debug flag switches to DEBUG level
Bug Note	Runtime-breaking bug: setup_mcp_server() receives parameter kalimind_client, but all 12 inner tool functions reference kali_client — an undefined name in that scope. This causes a NameError on every tool call. Fix: rename the parameter to kali_client, or replace all kali_client references inside setup_mcp_server() with kalimind_client.

2.4 kalimind.json — Claude Desktop Configuration

The kalimind.json file configures Claude Desktop to launch kalimind_mcp.py as an MCP server subprocess:

```
terminal
{
  "mcpServers": {
    "kalimind-server": {
      "command": "python3",
      "args": [
        "/absolute/path/to/kalimind_mcp.py",
        "--server",
        "http://LINUX_IP:5000/"
      ],
      "description": "KaliMind MCP Server",
      "timeout": 300,
      "alwaysAllow": []
    }
  }
}
```

Key deployment notes for kalimind.json:

- /absolute/path/to/kalimind_mcp.py: Must be replaced with the actual filesystem path where kalimind_mcp.py is installed on the operator's machine
- http://LINUX_IP:5000/: Must be replaced with the actual IP of the Kali VM (or 127.0.0.1 if using local SSH tunnel). This is passed as --server to kalimind_mcp.py
- alwaysAllow: []: Currently empty — all tool invocations prompt Claude Desktop for approval. Add tool names here to allow auto-execution without confirmation
- timeout: 300: Matches DEFAULT_REQUEST_TIMEOUT in kalimind_mcp.py

3. Flask API Endpoint Reference

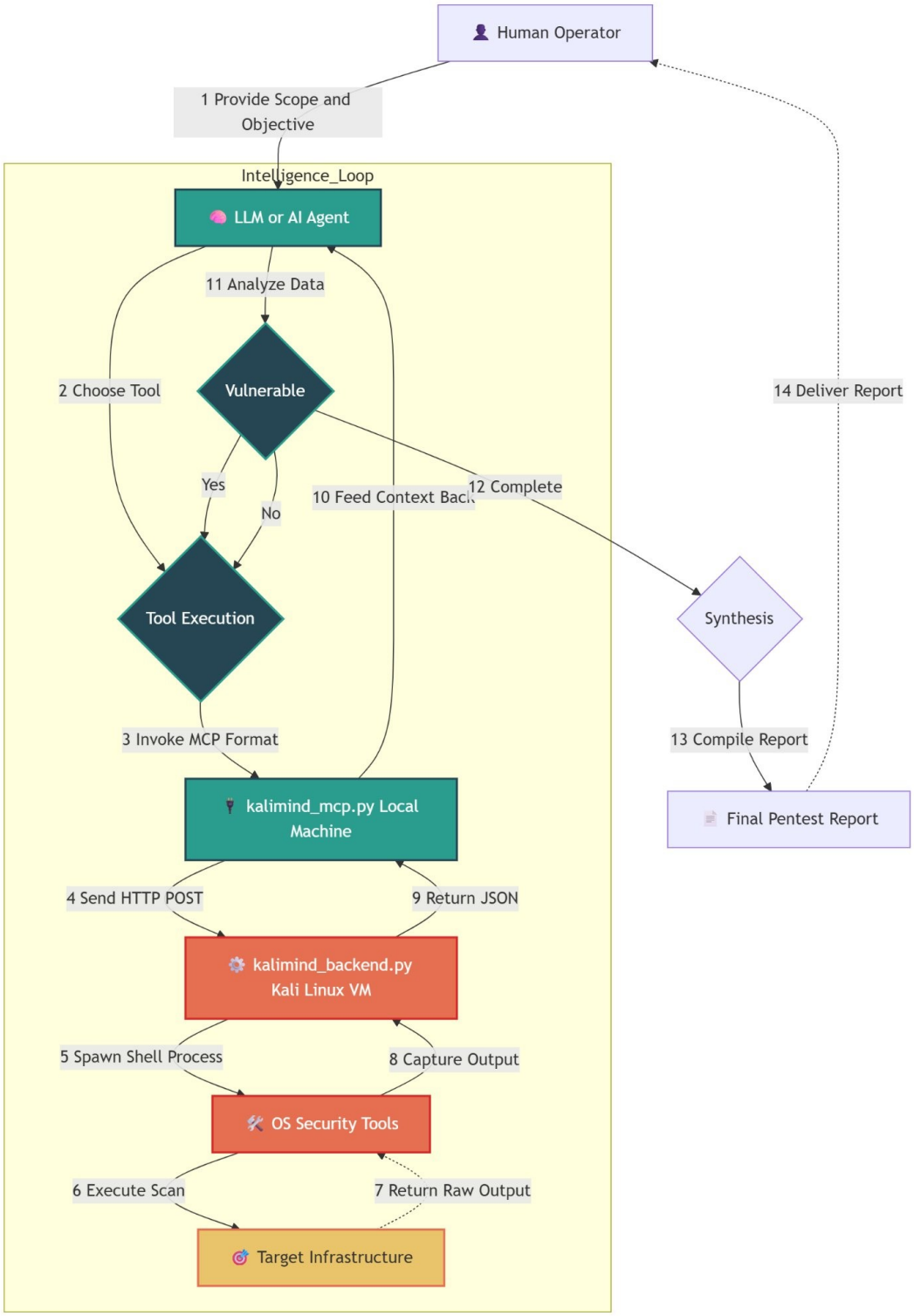
All endpoints are served by the Flask app on port 5000 (default). All POST endpoints accept application/json and return application/json.

Method	Endpoint	Parameters	Tool	Notes
GET	/health	—	which <tool>	Checks nmap, gobuster, dirb, nikto availability; returns JSON with tools_status{}
GET	/mcp/capabilities	—	—	Returns tool capability manifest (stub — Sprint 4 scope)
POST	/api/command	command: string	Any shell cmd	Generic raw command execution via CommandExecutor
POST	/api/tools/nmap	target, scan_type, ports, additional_args	nmap	Default: -sV scan, -T4 -Pn additional args
POST	/api/tools/gobuster	url, mode, wordlist, additional_args	gobuster	mode validated: dir|dns|fuzz|vhost; default wordlist: dirb/common.txt
POST	/api/tools/dirb	url, wordlist, additional_args	dirb	Default wordlist: dirb/common.txt
POST	/api/tools/nikto	target, additional_args	nikto	Runs: nikto -h <target>
POST	/api/tools/sqlmap	url, data, additional_args	sqlmap	Runs: sqlmap -u <url> --batch; POST data via --data=""
POST	/api/tools/metasploit	module, options{}	msfconsole	Auto-generates .rc resource script → msfconsole -q -r /tmp/mcp_msf_resource.rc
POST	/api/tools/hydra	target, service, username/file, password/file, additional_args	hydra	Runs: hydra -t 4 [-l|-L] [-p|-P] <target> <service>
POST	/api/tools/john	hash_file, wordlist, format, additional_args	john	Default wordlist: rockyou.txt; format is optional (auto-detect)
POST	/api/tools/wpscan	url, additional_args	wpscan	Runs: wpscan --url <url>
POST	/api/tools/enum4linux	target, additional_args	enum4linux	Default args: -a (full enumeration)
POST	/mcp/tools/kalimind_tools/<tool>	tool_name (URL param)	—	Direct tool execution (stub — Sprint 4 scope)

4. System Architecture

4.1 Architecture Overview

The following table represents KaliMind's end-to-end system architecture, showing all components, data flows, and the intelligence loop.



Layer / Actor	Description
Human Operator	Provides target scope and objectives via Claude Desktop. Receives the final penetration test report. Does not directly interact with any tools.
↓ (1) Scope & Objective	
LLM / AI Agent (Claude)	Intelligence Loop: plans attack strategy, invokes tools via @mcp.tool() calls, reads results, adapts approach, feeds context forward. Runs 10–50 tool calls per session.
↓ (2) MCP tool_use JSON	MCP protocol over stdio
kalimind_mcp.py (Operator's Local Machine)	FastMCP server: maps 11 tool names to HTTP POST calls. Uses KaliToolsClient.safe_post() / safe_get(). Default server: http://localhost:5000. Timeout: 300s.
↓ (3) HTTP POST localhost:5000	requests.post(json=data, timeout=300)
kalimind_backend.py (Kali Linux VM)	Flask REST API: dispatches each endpoint to CommandExecutor. Bind: 127.0.0.1:5000 by default. Timeout: 180s. 13 endpoints.
↓ (4) subprocess.Popen(shell=True)	Daemon threads stream stdout/stderr
OS Security Tools (Kali Linux)	nmap · gobuster · dirb · nikto · wpscan · sqlmap · msfconsole · hydra · john · enum4linux
↓ (5) Execute scan / exploit	→ (6) Return raw output
Target Infrastructure	In-scope hosts, ports, web apps, services (authorized targets only)
↑ (7) stdout/stderr → JSON → MCP tool_result → LLM context	Loop back for next tool decision
→ Synthesis	LLM compiles all findings → structured vulnerability report → Human Operator

5. Component Diagram

KaliMind's software components and their interfaces, as implemented in the codebase:

Component	Module/File	Interface Exposed	External Dependencies
MCP Client	kalimind_mcp.py	FastMCP stdio (11 @mcp.tool() handlers)	mcp, requests, Python 3.x
HTTP Client Layer	KaliToolsClient class	safe_get(), safe_post()	requests library
Flask Backend	kalimind_backend.py	HTTP REST API :5000 (13 endpoints)	Flask, Python 3.x
Subprocess Manager	CommandExecutor class	execute() → result dict	subprocess, threading
Tool Wrappers	Per-endpoint functions (nmap, gobuster, etc.)	Flask route handlers (POST)	OS CLI tools
Health Monitor	/health endpoint	GET /health → JSON	which <tool> subprocess
Metasploit Wrapper	/api/tools/metasploit	Generates /tmp/mcp_msf_resource.rc	msfconsole
LLM Agent	Claude (Anthropic API)	MCP tool_use / tool_result	Anthropic Claude API

Component	Module/File	Interface Exposed	External Dependencies
Config File	kalimind.json	Claude Desktop mcpServers config	Claude Desktop app

6. Use Case Diagram

Actor	Use Case	Code Reference
Human Operator	Define scan scope & target	Claude Desktop — natural language instruction
Human Operator	Receive final pentest report	LLM synthesis → structured report output
LLM Agent	Plan attack strategy	Intelligence_Loop in Claude reasoning context
LLM Agent	Invoke nmap_scan	@mcp.tool('nmap_scan') → POST /api/tools/nmap
LLM Agent	Invoke gobuster_scan / dirb_scan	@mcp.tool() → POST /api/tools/gobuster|dirb
LLM Agent	Invoke nikto_scan / wpscan_analyze	@mcp.tool() → POST /api/tools/nikto|wpscan
LLM Agent	Invoke sqlmap_scan	@mcp.tool() → POST /api/tools/sqlmap --batch
LLM Agent	Invoke metasploit_run	@mcp.tool() → POST /api/tools/metasploit → .rc script → msfconsole
LLM Agent	Invoke hydra_attack	@mcp.tool() → POST /api/tools/hydra -t 4
LLM Agent	Invoke john_crack	@mcp.tool() → POST /api/tools/john --wordlist
LLM Agent	Invoke enum4linux_scan	@mcp.tool() → POST /api/tools/enum4linux -a
LLM Agent	Execute raw command	@mcp.tool('execute_command') → POST /api/command
LLM Agent	Check server health	@mcp.tool('server_health') → GET /health
Kali Backend	Spawn tool subprocess	CommandExecutor.execute() — subprocess.Popen(shell=True)
Kali Backend	Stream output continuously	_read_stdout() / _read_stderr() daemon threads
Kali Backend	Apply timeout (180s)	process.wait(timeout=180) → SIGTERM → SIGKILL
Kali Backend	Return partial results on timeout	success=True if timed_out AND stdout/stderr captured

7. Sequence Diagram — Full Pentest Session

#	Human Operator	LLM Agent	kalimind_mcp.py	kalimind_backend.py	OS Tool	Target
1	→ Scope & target					
2		→ Plan attack				
3		→ nmap_scan(target				

#	Human Operator	LLM Agent	kalimind_mcp.py	kalimind_backend.py	OS Tool	Target
		, -sV)				
4			→ POST /api/tools/nmap			
5				→ Popen(nmap -sV ...)		
6					→ Scan ports	→
7				← stdout/stderr threads		
8			← JSON {stdout, success}			
9		← tool_result to context				
10		→ gobuster_scan (port 80 open)				
11–14		... repeat for nikto, sqlmap, hydra, metasploit, etc. ...				
15		→ Synthesize all findings				
16	← Final Report					

8. Deployment Diagram

Node 1: Operator's Local Machine	Node 2: Kali Linux VM / Docker Container
OS: Windows / macOS / Linux • Claude Desktop (Anthropic) • kalimind_mcp.py (FastMCP server subprocess) • Python 3.x + mcp + requests • kalimind.json → Claude Desktop config Connects to Kali VM via: localhost (if local) or SSH tunnel / VPN	OS: Kali Linux 2023+ | Port: 5000 (--ip 127.0.0.1) • kalimind_backend.py (Flask on 127.0.0.1:5000) • CommandExecutor (subprocess management) • Python 3.x + Flask + threading • nmap, gobuster, dirb, nikto, wpscan, sqlmap, msfconsole, hydra, john, enum4linux SECURITY: NEVER bind to 0.0.0.0 or public IP. Use --ip 127.0.0.1 (default).

9. Data Flow Diagram (DFD)

Level 1 — System DFD (per code implementation)

Process	Input Data	Output Data
P1: LLM Reasoning (Claude)	Scan scope + tool results from context window	MCP tool_use call: { tool_name, arguments{} }

Process	Input Data	Output Data
P2: MCP Translation (kalimind_mcp.py)	MCP tool_use JSON via stdio	HTTP POST JSON body to Flask :5000
P3: Request Dispatch (Flask routes)	HTTP POST with JSON params	Validated command string to CommandExecutor
P4: CommandExecutor	Shell command string	subprocess.Popen output capture
P5: Subprocess Threads	process.stdout / process.stderr pipes	Continuous line-by-line stdout_data / stderr_data
P6: Timeout Manager	process.wait(timeout=180)	SIGTERM or SIGKILL; return_code=-1
P7: Response Builder	stdout_data, stderr_data, return_code, timed_out	{ stdout, stderr, return_code, success, timed_out, partial_results }
P8: HTTP Response (Flask)	CommandExecutor result dict	jsonify(result) → HTTP 200 or 500
P9: Context Update (MCP)	JSON HTTP response	MCP tool_result fed to LLM context
P10: Report Synthesis	Complete LLM session context with all tool outputs	Structured penetration test report (Markdown/PDF)

10. Database Design — ER Diagram & Schema

Entities are derived from the data KaliMind produces and logs, aligned with the CommandExecutor response schema and tool invocation patterns.

Entity	Key Attributes	Relationships
ScanSession	session_id (PK), operator_id (FK), target_ip, target_scope, start_time, end_time, status	Has many: ToolInvocation; Has many: VulnerabilityFinding; Has one: PentestReport
ToolInvocation	invocation_id (PK), session_id (FK), tool_name [nmap|gobuster|dirb|nikto|sqlmap|metasploit|hydra|john|wpscan|enum4linux|command], command_string, stdout, stderr, exit_code, duration_ms, timed_out, partial_results, timestamp	Belongs to: ScanSession
VulnerabilityFinding	finding_id (PK), session_id (FK), invocation_id (FK), severity, title, description, evidence (stdout excerpt), recommendation, cvss_score	Belongs to: ScanSession; Derived from: ToolInvocation
PentestReport	report_id (PK), session_id (FK), generated_at, executive_summary, total_findings, critical_count, high_count, medium_count, low_count, report_format	Belongs to: ScanSession; Aggregates: VulnerabilityFinding
Operator	operator_id (PK), api_token_hash, created_at, last_active	Initiates: ScanSession
AuditLog	log_id (PK), operator_id (FK), session_id (FK), endpoint_called, command_string, timestamp, ip_address	References: Operator, ScanSession

11. Data Exchange Contract

11.1 Component Data Flows (from code)

Component Pair	Protocol / Mode	Frequency	Data Format	Timeout
Human Operator → LLM Agent	Claude Desktop (natural language)	1× per session	Free text	N/A
LLM Agent → kalimind_mcp.py	MCP tool_use JSON (stdio)	10–50× per session	{ tool, arguments{} }	300s (MCP client side)
kalimind_mcp.py → kalimind_backend.py	HTTP POST to localhost:5000	Same as above	JSON body	300s (requests timeout)
kalimind_backend.py → OS Tools	subprocess.Popen(shell=True)	Same as above	CLI string	180s (CommandExecutor)
OS Tools → Target Infrastructure	Network (TCP/UDP)	Per invocation	Network packets	Inherited from tool
OS Tools → kalimind_backend.py	subprocess PIPE (stdout/stderr threads)	Continuous streaming	Plain text lines	180s then SIGTERM→SIGKILL
kalimind_backend.py → kalimind_mcp.py	HTTP 200/500 JSON response	Same as above	{ stdout, stderr, return_code, success, timed_out, partial_results }	N/A
kalimind_mcp.py → LLM Agent	MCP tool_result JSON	Same as above	Structured tool result	N/A
LLM Agent → Synthesis	Internal LLM context	1× at end	Full session context	N/A
Synthesis → Human Operator	Text output / file export	1× per session	Markdown / PDF report	N/A

11.2 JSON Schemas — MCP Tool Call & Flask Response

Example: nmap_scan invocation from LLM → Flask response returned to LLM

```
terminal
// LLM → kalimind_mcp.py (MCP tool_use)
{ "tool": "nmap_scan",
  "arguments": { "target": "192.168.1.10", "scan_type": "-sV",
                 "ports": "1-1000", "additional_args": "-T4 -Pn" } }

// kalimind_mcp.py → kalimind_backend.py (HTTP POST /api/tools/nmap)
POST http://localhost:5000/api/tools/nmap
Content-Type: application/json
{ "target": "192.168.1.10", "scan_type": "-sV", "ports": "1-1000",
  "additional_args": "-T4 -Pn" }

// kalimind_backend.py builds and runs:  nmap -sV -p 1-1000 -T4 -Pn
192.168.1.10

// kalimind_backend.py → kalimind_mcp.py (HTTP 200 JSON)
{ "stdout": "PORT  STATE  SERVICE\n22/tcp open ssh\n80/tcp open http",
  "stderr": "", "return_code": 0, "success": true,
```

```
  "timed_out": false, "partial_results": false }

// kalimind_mcp.py → LLM Agent (MCP tool_result)
{ "tool_result": { ...above JSON... } }
```

11.3 Metasploit Special Flow

The Metasploit endpoint differs from other tools — it generates a resource script file rather than a direct command:

```
terminal
// Input to POST /api/tools/metasploit
{ "module": "exploit/unix/ftp/vsftpd_234_backdoor",
  "options": { "RHOSTS": "192.168.1.10", "LHOST": "192.168.1.5" } }

// Auto-generated /tmp/mcp_msf_resource.rc:
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.1.10
set LHOST 192.168.1.5
exploit

// Executed as: msfconsole -q -r /tmp/mcp_msf_resource.rc
// File cleaned up after execution via os.remove(resource_file)
```

12. Non-Functional Architecture Constraints

12.1 Performance (from code)

- Backend Timeout: COMMAND_TIMEOUT = 180 seconds (3 minutes) hardcoded in kalimind_backend.py. Configurable at class level.
- MCP Client Timeout: DEFAULT_REQUEST_TIMEOUT = 300 seconds (5 minutes) in kalimind_mcp.py, passed as requests timeout.
- Dual-Timeout Gap: The 120-second gap between backend (180s) and MCP client (300s) ensures the backend always kills the subprocess and returns a JSON response before the client times out waiting.
- Hydra Thread Limiting: hydra -t 4 limits to 4 parallel login attempts, preventing network flooding and staying within timeout.

12.2 Security (from code)

- Default Bind: kalimind_backend.py defaults to --ip 127.0.0.1. Never change to 0.0.0.0 in a non-isolated environment.
- No Auth in MVP: No authentication layer in current code. Sprint 4 scope: implement Bearer token on all Flask endpoints before production use.
- Shell Injection Risk: subprocess.Popen(shell=True) means all input parameters are passed directly to a shell. Gobuster validates mode against an allowlist [dir|dns|fuzz|vhost]. Other tools perform basic pass-through. Production: add input sanitization.
- Metasploit Resource File: Written to /tmp/mcp_msf_resource.rc and deleted after use. Risk: race condition if two concurrent Metasploit calls overwrite the same file. Production: use a UUID-based temp filename.

12.3 Reliability (from code)

- Partial Results: If a tool times out but produced output, success=True and partial_results=True — the LLM receives and reasons on partial data rather than a failure.

- Graceful Shutdown: SIGTERM sent first, then 5-second wait, then SIGKILL — prevents zombie processes.
- Health Check: GET /health checks all essential tools (nmap, gobuster, dirb, nikto) via which <tool>. Returns all_essential_tools_available: bool and per-tool status.
- Error Propagation: All exceptions caught in Flask routes and CommandExecutor; errors returned as JSON with error key — LLM can reason on errors and retry.

13. Conclusion

KaliMind's two-service microservices architecture successfully bridges LLM reasoning with Kali Linux tooling through a clean separation of concerns. The MCP client layer keeps the LLM isolated from direct shell access, while the Flask backend's CommandExecutor provides robust subprocess management with dual-timeout protection (180s backend / 300s MCP client), graceful SIGTERM→SIGKILL handling, and partial-result recovery — ensuring the intelligence loop continues even when individual tools time out.

The system is extensible by design — new tools require only a Flask route and an MCP decorator — and the database schema aligns directly with the CommandExecutor response structure, minimising implementation effort. Remaining Sprint 4 items (Bearer token auth, input sanitisation, UUID-based Metasploit resource files, and the kali_client naming fix) are well-scoped and non-breaking. KaliMind is ready for controlled lab deployment and serves as a strong foundation for production hardening under the BTech CSE (Cyber Security) capstone at SRM Institute of Science and Technology.